

Manual de Usuario

Guía operativa para configurar, desplegar y operar el Gateway HTTP Modular: reverse proxy sobre Undici, middleware pipeline (rate-limit, CORS, JWT, circuit breaker), hot-reload por SIGHUP, métricas Prometheus y agregación de health.

Resumen ejecutivo

El Gateway HTTP Modular es un reverse proxy escrito en TypeScript sobre Fastify + Undici, pensado como capa única de entrada para arquitecturas basadas en microservicios. Reúne las capacidades comunes que toda organización termina reimplementando (cors, rate-limit, autenticación JWT, circuit breaker, métricas, health) detrás de un único archivo de configuración declarativo, con hot-reload sin downtime y trazabilidad operativa completa.

Qué problema resuelve

- **Punto único de entrada configurable** para microservicios, frontends y backends legacy, sin reescribir cada servicio.
- **Seguridad transversal**: rate-limit por IP/ventana, CORS por origen, validación JWT contra JWKS remoto, circuit breaker para aislar backends caídos.
- **Observabilidad**: endpoint `/metrics` en Prometheus, endpoint agregado `/health`, logs estructurados en JSON.
- **Operación sin downtime**: reload por `SIGHUP` de rate-limits, CORS, JWT y niveles de log, sin reconectar a Redis ni reiniciar Fastify.

Stack técnico

Componente	Tecnología	Propósito
Runtime	Node.js 20+ / TypeScript	Servicio standalone con build ESM
Servidor HTTP	Fastify 5.x	Framework de routing y hooks
Cliente upstream	Undici 6.x	Pool HTTP/1.1 y HTTP/2 de alto rendimiento
Rate limiting	Redis 7+ (ioredis)	Contadores fijos por ventana temporal
JWT	jose	Validación HS256/HS384/HS512 y JWKS RS256
Logs	Pino 10.x	Logging estructurado JSON con redacción
Métricas	prom-client	Endpoint <code>/metrics</code> Prometheus
Validación config	Zod	Schema tipado con fall-fast al startup

Para quién es este manual

- **Operadores / SRE** que despliegan y monitorean el gateway en producción.
- **Desarrolladores backend** que exponen nuevos servicios detrás del gateway.

- **Integradores** que ajustan CORS, JWT y políticas de rate-limit por tenant.

Contenido

1. Inicio rápido
2. Anatomía del archivo de configuración
3. Servidor y red
4. Redis para rate limiting
5. Logging
6. Health Aggregation
7. Rutas: el reverse proxy
8. Rate limiting y overrides
9. CORS
10. Autenticación JWT
11. Circuit breaker y reintentos
12. Métricas Prometheus
13. Hot reload (SIGHUP)
14. Despliegue con Docker
15. Solución de problemas
16. Anexo: hoja de referencia rápida

1. Inicio rápido

Esta sección asume un entorno local Linux/macOS con Node.js 20+ y un Redis accesible en `localhost:6379`. Si solo quieres ver el Gateway funcionando en menos de 5 minutos, sigue el camino directo.

Paso 1 — Instalar dependencias

El proyecto usa **pnpm** exclusivamente. Nunca uses `npm` ni `npx` por convención del repositorio.

```
# Desde la raíz del proyecto
pnpm install
```

Paso 2 — Levantar Redis

Puedes usar un Redis nativo, Docker, o el `docker-compose` de la plantilla en `docker/docker-compose.example.yml`:

```
# Con docker
docker run -d --name gateway-redis -p 6379:6379 redis:7-alpine

# O desde la plantilla (incluye también un backend mock, Dozzle, Prometheus y Grafana)
docker compose -f docker/docker-compose.example.yml up -d redis-cache
```

Paso 3 — Definir la variable de entorno obligatoria

```
# Linux / macOS
export REDIS_URL="redis://localhost:6379"

# Windows PowerShell
$env:REDIS_URL = "redis://localhost:6379"
```

Paso 4 — Arrancar el gateway en modo desarrollo

```
pnpm dev
```

El comando ejecuta `tsx watch src/index.ts` y el gateway queda escuchando en `http://0.0.0.0:3000` con hot-reload de TypeScript ante cualquier cambio en `src/`.

Paso 5 — Probar que funciona

```
# Endpoint de salud agregado
curl http://localhost:3000/health

# Métricas Prometheus
curl http://localhost:3000/metrics

# Ruta /api definida en gateway.yaml – proxea a http://localhost:8080
curl http://localhost:3000/api/usuarios
```

LISTO

Si los tres comandos respondieron correctamente tienes el gateway operativo con configuración por defecto. Continúa con el capítulo 2 para entender qué controla cada sección de `gateway.yaml`.

Comandos útiles del día a día

Comando	Propósito
<code>pnpm dev</code>	Desarrollo con watch de TypeScript
<code>pnpm build</code>	Compila a <code>dist/</code> (producción)
<code>pnpm start</code>	Arranca el binario compilado
<code>pnpm test</code>	Suite completa (Vitest)
<code>pnpm test:unit</code>	Solo tests unitarios
<code>pnpm test:integration</code>	Solo tests de integración
<code>pnpm lint</code>	Lint sobre <code>src/</code> y <code>tests/</code>
<code>pnpm format</code>	Formatea con Prettier
<code>pnpm docker:build</code>	Build de la imagen Docker del gateway

2. Anatomía del archivo de configuración

Toda la configuración del gateway vive en un único archivo YAML: `config/gateway.yaml`. Su ruta puede sobrescribirse con la variable de entorno `CONFIG_PATH`. El archivo es validado por Zod al arrancar: cualquier error produce un mensaje claro y el proceso aborta con exit code 1 (fail-fast).

Esqueleto mínimo

```
# config/gateway.yaml

server:          # Puerto y host del gateway
  port: 3000
  host: "0.0.0.0"

redis:           # Conexión y comportamiento ante fallos
  url: "${REDIS_URL}"
  onFailure: "open"

logging:
  level: "info"

routes:          # Mínimo 1 ruta requerida
  - prefix: "/api"
    target: "http://localhost:8080"
```

Top-level: qué controla cada bloque

Bloque	Función	Capítulo
<code>server</code>	Puerto y bind del servidor Fastify	3
<code>redis</code>	Conexión al Redis usado por rate-limit	4
<code>logging</code>	Nivel de log Pino	5
<code>health</code>	Endpoint agregado de health	6
<code>routes</code>	Reverse proxy: prefijos y backends	7
<code>overrides</code>	Rate-limit por path exacto	8
<code>cors</code> + <code>corsOverrides</code>	Política CORS global	9
<code>jwt</code> + <code>jwtOverrides</code>	Validación JWT (JWKS o shared secret)	10
<code>metrics</code>	Endpoint Prometheus y labels	12

INTERPOLACIÓN DE VARIABLES

Cualquier valor escalar acepta la sintaxis `${NOMBRE_VARIABLE}`. Si la variable no está definida, el arranque falla con `MissingEnvVarError`. Es la forma recomendada de manejar secretos en pipelines CI/CD.

3. Servidor y red

Define en qué puerto y dirección escucha el gateway. Cualquier cambio en este bloque **requiere reinicio** (no es hot-reloadable) porque implica re-bindear el socket TCP.

Esquema

Campo	Tipo	Por defecto	Descripción
<code>port</code>	number 1–65535	3000	Puerto TCP donde escucha Fastify
<code>host</code>	string	<code>0.0.0.0</code>	Interfaz de bind. Use <code>127.0.0.1</code> para localhost únicamente

Ejemplo — desarrollo local

```
server:
  port: 3000
  host: "127.0.0.1"
```

Ejemplo — dentro de contenedor Docker

```
server:
  port: ${PORT}           # viene de la variable de entorno del contenedor
  host: "0.0.0.0"         # obligatorio para aceptar tráfico desde fuera del container
```

Ejemplo — producción detrás de un load balancer

```
server:
  port: 8080
  host: "0.0.0.0"
```

Cuando el gateway está detrás de un ALB/Nginx, los headers `X-Forwarded-For`, `X-Forwarded-Proto` y `X-Forwarded-Host` se reenvían al upstream para que los servicios vean la IP real del cliente. El rate-limiter usa siempre `request.ip` que Fastify resuelve correctamente con la cabecera adecuada.

CUIDADO

Si modificas `server.port` o `server.host` en un archivo ya cargado, el gateway lo registra como `warn` y lo ignora hasta el próximo reinicio. Usa `SIGHUP` solo para cambios hot-reloadables (ver [capítulo 13](#)).

4. Redis para rate limiting

El gateway usa Redis como backend de contadores para rate limiting. Es la única dependencia externa obligatoria y se conecta al arrancar con hasta tres reintentos espaciados un segundo; si los tres fallan el proceso aborta.

Esquema

Campo	Tipo	Por defecto	Descripción
<code>url</code>	string	—	URL completa tipo <code>redis://host:puerto</code> o <code>rediss://</code> para TLS
<code>onFailure</code>	enum	<code>open</code>	Comportamiento si Redis falla en runtime: <code>open</code> deja pasar, <code>closed</code> responde 503

Ejemplo — Redis local sin autenticación

```
redis:
  url: "${REDIS_URL}"
  onFailure: "open"
```

```
# .env o exportación manual
REDIS_URL="redis://localhost:6379"
```

Ejemplo — Redis con autenticación por password

```
redis:
  url: "redis://:${REDIS_PASSWORD}@redis-prod.internal:6379/0"
  onFailure: "closed"
```

Ejemplo — Redis en clúster con TLS

```
redis:
  url: "rediss://cache-cluster.example.com:6380"
  onFailure: "open"
```

Decidir entre `open` y `closed`

ONFAILURE: OPEN

Si Redis se cae, las requests **pasan sin rate-limit**. Prioriza disponibilidad sobre control de abuso. Útil para servicios internos donde un atacante ya está dentro del perímetro.

ONFAILURE: CLOSED

Si Redis se cae, todas las requests devuelven `503 Service Unavailable`. Prioriza protección. Útil cuando el rate-limit es contractual (tiers de API de pago) y tolerar abuso es peor que perder tráfico.

BUENA PRÁCTICA

En producción usa siempre `onFailure: closed` con un Redis gestionado de alta disponibilidad (ElastiCache, Memorystore, Upstash). Configurar fall-open por error operacional suele terminar abriendo la puerta a abuso masivo justo cuando más necesitas protección.

5. Logging

El logger es Pino, con salida en JSON y redacción automática de cabeceras sensibles. Los secretos (`authorization` , `cookie` , `proxy-authorization` , `set-cookie`) nunca aparecen en los logs.

Esquema

Campo	Valores aceptados	Default
<code>level</code>	<code>debug</code> · <code>info</code> · <code>warn</code> · <code>error</code>	<code>info</code>

Ejemplo — producción con nivel info

```
logging:
  level: "info"
```

Ejemplo — diagnóstico puntual

```
logging:
  level: "debug"
```

`debug` añade trazas de decisiones CORS, transición de circuitos, refresh de JWKS, etc. Es ruidoso y no apto para producción continua. Se cambia en caliente con `SIGHUP` .

EJEMPLO DE LOG

```
{
  "level": 30,
  "time": "2026-07-04T21:24:18.812Z",
  "cors": { "decision": "allowed", "origin": "https://app.flashdrop.cl" },
  "route": "/api/orders",
  "reqId": "3c4f-...",
  "msg": "cors: allowed origin"
}
```

6. Health Aggregation

El endpoint `GET /health` consulta en paralelo el `backendPath` de cada backend declarado en `routes[]` y devuelve un JSON agregado con el estado por servicio. Está pensado para readiness probes de Kubernetes, load balancers externos y dashboards de observabilidad.

Esquema

Campo	Tipo	Default	Descripción
<code>enabled</code>	boolean	<code>true</code>	Si está en <code>false</code> el endpoint devuelve 404
<code>path</code>	string	<code>/health</code>	Path donde el gateway expone el agregado
<code>backendPath</code>	string	<code>/health</code>	Path a consultar en cada backend downstream
<code>timeoutMs</code>	int > 0	<code>2000</code>	Timeout por servicio; cumplido el plazo se marca como <code>down</code>

Mapeo de estados

Respuesta del backend	Estado por servicio	Estado global	Código HTTP
2xx / 3xx	<code>ok</code>	<code>down</code> si alguno cae, <code>degraded</code> si no hay ninguno caído pero sí <code>degraded</code> , si no <code>ok</code>	<code>503</code> si global down, <code>200</code> en cualquier otro caso
4xx	<code>degraded</code>		
5xx, timeout, ECONNREFUSED	<code>down</code>		

Ejemplo A — configuración estándar

```
health:
  enabled: true
  path: "/health"
  backendPath: "/health"
  timeoutMs: 2000
```

Ejemplo B — health deshabilitado (para entornos efímeros)

```
health:
  enabled: false
```

Ejemplo C — path personalizado por entorno

```
health:
  enabled: true
  path: "/status"           # endpoint expuesto por el gateway
  backendPath: "/internal/health" # endpoint de cada microservicio
  timeoutMs: 1500
```

Ejemplo de respuesta

```
{
  "status": "degraded",
  "timestamp": "2026-07-04T21:24:18.812Z",
  "services": [
    {
      "name": "orders-service",
      "status": "ok",
      "statusCode": 200,
      "latencyMs": 18
    },
    {
      "name": "catalog-service",
      "status": "down",
      "latencyMs": 2000,
      "error": "timeout after 2000ms"
    }
  ]
}
```

RESOLUCIÓN DE NOMBRES

El campo `name` de cada servicio se resuelve como `route.backendName ?? hostname(target) ?? route.prefix`. Si necesitas un nombre legible en Prometheus y Grafana, define `backendName` en cada ruta.

El endpoint de health NO consume rate-limit, no requiere autenticación, no pasa por el pipeline de proxy y se excluye de las métricas `gateway_http_*`. Las decisiones operativas sobre el estado global son responsabilidad del cliente que consulta (load balancer, Kubernetes).

7. Rutas: el reverse proxy

El bloque `routes[]` es el corazón del gateway. Cada elemento declara una regla de reenvío: cuando llega una request a un path que empieza con `prefix`, se redirige a `target` usando Undici como cliente HTTP.

Reglas de matching

1. **Exact override:** si el path coincide con un `overrides[].path`, gana sobre cualquier ruta.
2. **Longest prefix:** si hay varias rutas con prefijos coincidentes, gana la más larga.
3. **Sin coincidencia:** el gateway responde `404 Not Found` (excepto `/metrics` y `/health` que están reservados).

Esquema

Campo	Tipo	Requerido	Descripción
<code>prefix</code>	string	sí	Prefijo de la ruta; debe comenzar con <code>/</code> y no terminar con <code>/</code> (excepto la raíz <code>/</code>)
<code>target</code>	URL http(s)	sí	URL del backend upstream
<code>stripPrefix</code>	boolean	no	Si es <code>true</code> , el prefijo se elimina del path antes de enviar al upstream
<code>rateLimit</code>	objeto	no	Política de rate-limit por IP
<code>timeout</code>	objeto	no	Tiempos máximos de espera por fase de la conexión upstream
<code>jwt</code>	objeto	no	Política de autenticación JWT
<code>circuitBreaker</code>	objeto	no	Política de circuit breaker y retries
<code>cors</code>	objeto	no	Override parcial de CORS para esta ruta
<code>retryableMethods</code>	string[]	no	Métodos HTTP que se reintentan ante fallo de red
<code>metricsLabel</code>	string	no	Etiqueta de baja cardinalidad en métricas
<code>backendName</code>	string	no	Nombre lógico del backend para health aggregation

Ejemplo A — el mínimo viable

```
routes:
  - prefix: "/api"
    target: "http://localhost:8080"
```

Toda request a `http://gateway/api/usuarios` se reenvía a `http://localhost:8080/api/usuarios`.

Ejemplo B — `stripPrefix` para reescritura

```
routes:
  - prefix: "/admin"
    target: "http://localhost:9090"
    stripPrefix: true
```

`GET /admin/usuarios` → `GET http://localhost:9090/usuarios`.

Ejemplo C — múltiples microservicios con timeouts diferenciados

```
routes:
  # Auth — rápido
  - prefix: "/api/auth"
    target: "http://auth-service:8082"
    stripPrefix: true
    backendName: "auth"
    timeout:
      connect: 1000
      headers: 2000
      body: 5000

  # Reports — más permisivo
  - prefix: "/api/reports"
    target: "http://reports-service:8087"
    stripPrefix: true
    backendName: "reports"
    timeout:
      connect: 3000
      headers: 5000
      body: 120000 # 2 min para generación de PDFs
```

Ejemplo D — añadir una API externa (PokeAPI)


```
routes:
  - prefix: "/api/v2"
    target: "https://pokeapi.co"
    stripPrefix: false
    metricsLabel: "pokeapi"
    backendName: "pokeapi-ext"
    ratelimit:
      maxRequests: 8           # agresivo para evitar ban de la API pública
      windowSeconds: 60
```

CAMBIOS ESTRUCTURALES REQUIEREN REINICIO

Modificar `prefix`, `target`, `stripPrefix` o `timeout` requiere reinicio del proceso. Agregar o quitar rutas también. Usa `SIGHUP` solo para `ratelimit`, `cors`, `jwt`, `overrides`, `corsOverrides`, `jwtOverrides` y `logging.level`.

8. Rate limiting y overrides

El rate limiter implementa una ventana fija por IP usando ioredis como pipeline atómico. El motor es conservador en memoria y la latencia añadida es despreciable (<1 ms p99 por request).

Esquema de `rateLimit`

Campo	Tipo	Descripción
<code>maxRequests</code>	<code>int > 0</code>	Número máximo de requests dentro de la ventana
<code>windowSeconds</code>	<code>int > 0</code>	Tamaño de la ventana en segundos

Ejemplo A — por ruta

```
routes:
  - prefix: "/api"
    target: "http://backend:3000"
    rateLimit:
      maxRequests: 1000
      windowSeconds: 60  # 1000 req/min por IP
```

Ejemplo B — límites diferenciados por sensibilidad

```
routes:
  - prefix: "/api/public"
    target: "http://public-svc:8086"
    rateLimit:
      maxRequests: 2000
      windowSeconds: 60

  - prefix: "/api/admin"
    target: "http://admin-svc:8088"
    rateLimit:
      maxRequests: 200      # más estricto para endpoints administrativos
      windowSeconds: 60

  - prefix: "/api/health"
    target: "http://health-svc:8087"
    # sin rateLimit – no se especifica, no se aplica
```

Overrides por path exacto

A veces una ruta entera es permisiva pero un endpoint concreto requiere control estricto (ej: `/api/login` para evitar fuerza bruta). Los `overrides` ganan sobre la política de la ruta y son **hot-reloadables**.

Esquema de `overrides[]`

```
overrides:
  - path: "/api/login"      # path exacto, debe empezar con /
    rateLimit:
      maxRequests: 5        # 5 intentos por minuto por IP
      windowSeconds: 60
```

Ejemplo C — protección contra fuerza bruta de login

```
overrides:
  - path: "/api/auth/login"
    rateLimit:
      maxRequests: 5
      windowSeconds: 60

  - path: "/api/auth/2fa"
    rateLimit:
      maxRequests: 3
      windowSeconds: 300    # 3 intentos cada 5 minutos

  - path: "/api/auth/register"
    rateLimit:
      maxRequests: 10
      windowSeconds: 3600   # 10 registros por hora
```

Ejemplo D — endpoint de búsqueda costoso

```
routes:
  - prefix: "/api"
    target: "http://backend:3000"
    rateLimit:
      maxRequests: 1000
      windowSeconds: 60

overrides:
  - path: "/api/search"
    rateLimit:
      maxRequests: 10      # búsqueda es cara, bajamos a 10 por minuto
      windowSeconds: 60
```

COMPORTAMIENTO AL EXCEDER EL LÍMITE

Cuando se alcanza `maxRequests` dentro de la ventana, el gateway responde `429 Too Many Requests` con el siguiente cuerpo:

```
{
  "statusCode": 429,
  "error": "Too Many Requests",
  "message": "Rate limit exceeded: 5 requests per 60 seconds",
  "requestId": "3c4f-..."
}
```

9. CORS

El plugin CORS se ejecuta **primero** en el pipeline, antes de rate-limit, JWT y circuit breaker. Esto garantiza que los preflights (OPTIONS con `Origin`) no consuman recursos compartidos y que los preflights respondan incluso si el circuit breaker del backend está abierto.

Precedencia (de mayor a menor)

1. `corsOverrides[path=X]` — path exacto
2. `routes[].cors` — prefijo de la ruta
3. `cors` — global
4. CORS deshabilitado — si nada está definido

Esquema

Campo	Tipo	Default	Descripción
<code>enabled</code>	boolean	false	Activa/desactiva CORS para el nivel
<code>origins</code>	string[]	[]	Lista de orígenes permitidos o <code>["*"]</code>
<code>methods</code>	string[]	GET,POST,PUT,DELETE,PATCH,OPTIONS,HEAD	Métodos HTTP permitidos
<code>allowedHeaders</code>	string[]	[Content-Type,Authorization]	Cabeceras que el cliente puede enviar
<code>exposedHeaders</code>	string[]	[]	Cabeceras expuestas al JS del cliente (solo en respuestas reales)
<code>credentials</code>	boolean	false	Permite envío de credenciales (cookies, Authorization)
<code>maxAge</code>	int > 0	86400	Tiempo de caché del preflight en segundos (24h)

Escenario 1 — Básico. Frontend público contra API interna

BÁSICO

```

cors:
  enabled: true
  origins:
    - "https://app.flashdrop.cl"
    - "https://admin.flashdrop.cl"
  methods:
    - GET
    - POST
    - PUT
    - DELETE
    - PATCH
    - OPTIONS
  allowedHeaders:
    - Content-Type
    - Authorization
    - X-Request-ID
  exposedHeaders:
    - X-Request-ID
    - X-RateLimit-Remaining
  credentials: false
  maxAge: 86400

```

Toda ruta hereda esta política global salvo override.

Escenario 2 — Override por ruta. `/api/dev` abierto a cualquier origen

PRODUCCIÓN

```

routes:
  # Hereda CORS global
  - prefix: "/api"
    target: "http://backend:3000"
    stripPrefix: true

  # Override SOLO para /api/dev - wildcard
  - prefix: "/api/dev"
    target: "http://backend-dev:3000"
    stripPrefix: true
    cors:
      origins:
        - "*"
      methods:
        - "*"

```

Escenario 3 — Credenciales activas para una ruta de auth

PRODUCCIÓN

```

routes:
  - prefix: "/api/auth"
    target: "http://auth-service:8082"
    stripPrefix: true
    cors:
      origins:
        - "https://app.flashdrop.cl"
      credentials: true
      allowedHeaders:
        - Content-Type
        - Authorization
        - Cookie

```

Escenario 4 — Override por path exacto. Login completamente abierto

AVANZADO

```

# corsOverrides tiene prioridad máxima
corsOverrides:
  - path: "/api/auth/login"
    cors:
      origins:
        - "*"
      credentials: false

```

Escenario 5 — Deshabilitar CORS para una ruta interna

AVANZADO

```

routes:
  - prefix: "/internal"
    target: "http://internal-service:9000"
    stripPrefix: true
    cors:
      enabled: false

```

Validación al startup (fall-fast)

Las siguientes combinaciones se rechazan al arrancar con mensaje claro:

Combinación	Error
<code>enabled: true</code> con <code>origins: []</code>	<i>cors: enabled=true requires at least one origin</i>
<code>credentials: true</code> con <code>origins: ["*"]</code>	<i>cors: cannot use credentials=true with wildcard origin "*"</i>
<code>credentials: true</code> con <code>allowedHeaders: ["*"]</code>	<i>cors: cannot use allowedHeaders=["*"] with credentials=true</i>

PREFLIGHT CORTO-CIRCUITO

Un `OPTIONS` con cabecera `Origin` permitido se responde directamente con `204 No Content` y los headers CORS correspondientes. El backend no recibe la request.

10. Autenticación JWT

El gateway soporta dos modos de validación JWT que pueden coexistir en distintas rutas. Si una ruta no incluye el bloque `jwt`, es pública (no se valida ningún token).

Dos modos

Modo	Algoritmos	Cuándo usarlo
shared-secret	HS256, HS384, HS512	Compatibilidad hacia atrás, integraciones internas, baja rotación de claves. Cada ruta lleva su propio <code>secret</code> . No requiere sección global <code>jwt</code> .
jwt	RS256	Producción. Las claves privadas quedan en el Auth Service, el gateway descubre las públicas vía JWKS endpoint (RFC 7517) y rota automáticamente. Soporta múltiples issuers.

Precedencia

1. `jwtOverrides[path=X]` — path exacto
2. `routes[].jwt` — por ruta
3. `jwt` (global) — usado por rutas JWKS

Modo 1 · shared-secret (HS256)

BÁSICO

Cada ruta declara su secreto y algoritmo localmente.

```
routes:
- prefix: "/api/protected"
  target: "http://backend:3000"
  stripPrefix: true
  jwt:
    enabled: true
    secret: "${JWT_SECRET}"
    algorithm: "HS256"
    issuer: "flashdrop-api"           # claim iss esperado
    audience: "flashdrop-api"        # claim aud esperado
    forwardClaims:
      - sub
      - role
      - email
```

Los `forwardClaims` se reenvían al backend en cabeceras `x-jwt-claim-<nombre>`, útil para que el servicio downstream sepa quién es el usuario sin re-validar el token.

Ruta explícitamente pública dentro de un bloque protegido

```
routes:
  - prefix: "/api/auth/login"
    target: "http://auth:8082"
    stripPrefix: true
  jwt:
    enabled: false           # Login no requiere token
```

Modo 2 · JWKS (RS256 contra Auth Service)

PRODUCCIÓN

Declara los issuers una sola vez en la sección global `jwt`; las rutas referencian por nombre. `issuer: any` acepta tokens de cualquiera de los issuers registrados (mapeo por `iss` claim).

Esquema de `jwt.issuers[]`

Campo	Tipo	Default	Descripción
<code>name</code>	string	—	Identificador único del issuer
<code>jwksUri</code>	URL https	—	Endpoint que sirve JWKS (RFC 7517)
<code>issuer</code>	string	—	Valor esperado del claim <code>iss</code>
<code>audience</code>	string	opcional	Valor esperado del claim <code>aud</code>
<code>cacheTtlSeconds</code>	int	3600	Tiempo de vida de la caché en segundos
<code>staleGracePeriodSeconds</code>	int	1800	Ventana de gracia post-TTL donde se sigue sirviendo caché si falla el refresh
<code>refreshCooldownSeconds</code>	int	30	Mínimo entre refreshes síncronos
<code>refreshOnMiss</code>	boolean	true	Si un <code>kid</code> desconocido dispara refresh síncrono
<code>timeoutMs</code>	int	3000	Timeout HTTP al endpoint JWKS

Ejemplo A — issuer único en producción

```

jwt:
  enabled: true
  mode: "jwks"
  issuers:
    - name: "auth-service-prod"
      jwksUri: "https://auth.flashdrop.cl/.well-known/jwks.json"
      issuer: "https://auth.flashdrop.cl"
      audience: "flashdrop-api"

routes:
  - prefix: "/api/orders"
    target: "http://orders-service:8084"
    jwt:
      issuer: "auth-service-prod"  # referencia por nombre

```

Ejemplo B — multi-tenant con dos issuers

```

jwt:
  enabled: true
  mode: "jwks"
  issuers:
    - name: "tenant-a"
      jwksUri: "https://auth.tenant-a.com/.well-known/jwks.json"
      issuer: "https://auth.tenant-a.com"
    - name: "tenant-b"
      jwksUri: "https://auth.tenant-b.com/.well-known/jwks.json"
      issuer: "https://auth.tenant-b.com"

routes:
  # Ruta de tenant-a: solo tokens de su issuer
  - prefix: "/api/tenant-a"
    target: "http://tenant-a-api:8084"
    jwt:
      issuer: "tenant-a"

  # Ruta mixta: acepta tokens de cualquier issuer registrado
  - prefix: "/api/public"
    target: "http://public-svc:8086"
    jwt:
      issuer: "any"

```

Ejemplo C — cache corta para staging con rotación frecuente

```

jwt:
  mode: "jwks"
  issuers:
    - name: "auth-staging"
      jwksUri: "https://auth-staging.flashdrop.cl/.well-known/jwks.json"
      issuer: "https://auth-staging.flashdrop.cl"
      cacheTtlSeconds: 600      # 10 minutos – rotación más agresiva
      staleGracePeriodSeconds: 300 # 5 min de gracia

```

Códigos de error

Código	Cuándo
401 Unauthorized	Token ausente, firma inválida, expirado, claims incorrectos, <code>kid</code> desconocido, <code>iss</code> / <code>aud</code> no coinciden
503 Service Unavailable	El endpoint JWKS no responde más allá de <code>staleGracePeriodSeconds</code> y no hay claves en caché

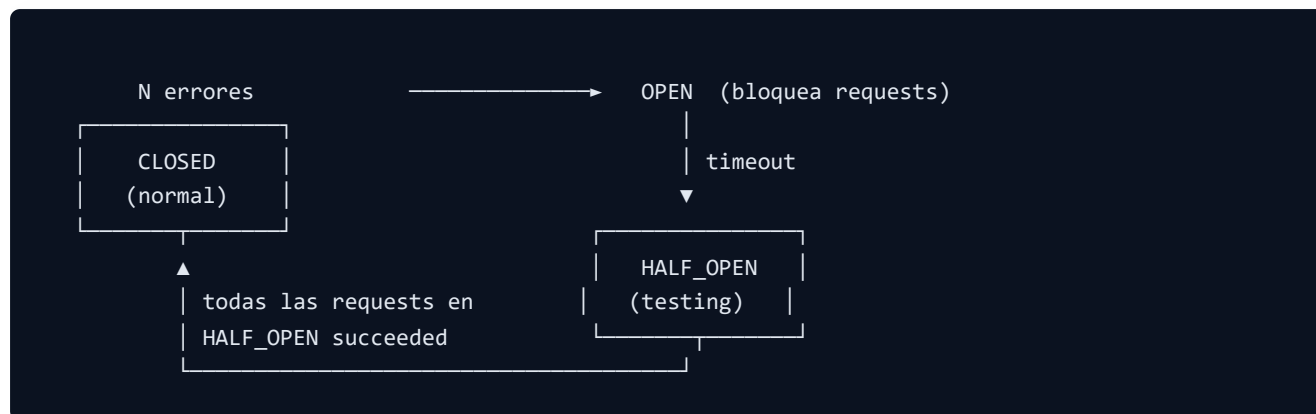
Métricas Prometheus disponibles

Métrica	Tipo	Labels	Descripción
<code>gateway_jwt_validations_total</code>	Counter	<code>result</code>	<code>ok</code> , <code>missing_token</code> , <code>unknown_kid</code> , <code>missing_kid</code> , <code>expired</code> , <code>invalid_issuer</code> , <code>invalid_audience</code> , <code>invalid_claims</code> , <code>invalid_signature</code> , <code>service_unavailable</code>
<code>gateway_jwks_refresh_total</code>	Counter	<code>result</code>	<code>ok</code> , <code>error</code> , <code>cooldown</code>

11. Circuit breaker y reintentos

El circuit breaker protege cada backend de sobrecargas aislando fallas en cascada. Cuando un upstream acumula errores, deja de recibir tráfico durante un tiempo de recuperación, prueba con un subconjunto y, si va bien, vuelve a la operación normal.

Diagrama de estados



Esquema de `circuitBreaker`

Campo	Tipo	Default	Descripción
<code>enabled</code>	boolean	true	Activa/desactiva el circuit breaker
<code>errorThreshold</code>	1–100	50	% de errores en la ventana para abrir
<code>requestCount</code>	int > 0	100	Tamaño de la ventana de requests
<code>recoveryTimeMs</code>	int > 0	30000	Tiempo en OPEN antes de pasar a HALF_OPEN
<code>halfOpenRequests</code>	int > 0	3	Requests exitosas requeridas en HALF_OPEN para volver a CLOSED
<code>maxRetries</code>	int ≥ 0	3	Reintentos por request antes de propagar el error
<code>retryDelayMs</code>	int > 0	100	Base para el cálculo de backoff exponencial
<code>retryMaxDelayMs</code>	int > 0	5000	Tope máximo del delay por reintento

Ejemplo A — circuit breaker ajustado para backend transaccional

```

routes:
- prefix: "/api/orders"
  target: "http://orders-backend:8080"
  circuitBreaker:
    enabled: true
    errorThreshold: 30      # 30% es suficiente para abrir
    requestCount: 50
    recoveryTimeMs: 60000  # 1 minuto antes de reintentar
    halfOpenRequests: 5
    maxRetries: 2
    retryDelayMs: 200
    retryMaxDelayMs: 10000

```

Ejemplo B — circuit breaker permisivo para contenido estático

```

routes:
- prefix: "/api/public"
  target: "http://public-backend:8080"
  circuitBreaker:
    enabled: true
    errorThreshold: 70      # tolera más errores antes de aislar
    requestCount: 200
    recoveryTimeMs: 15000  # recuperación rápida

```

Ejemplo C — deshabilitado para una ruta interna

```

routes:
- prefix: "/internal/jobs"
  target: "http://internal-svc:9000"
  circuitBreaker:
    enabled: false

```

Reintentos (retries)

Los reintentos usan exponential backoff con full jitter para evitar "thundering herd" cuando un backend se recupera. El cálculo del delay es:

```

// attempt = número de reintentos (0 = primer reintentos)
delay = min(retryDelayMs * 2^attempt, retryMaxDelayMs)
jitter = random(0, delay)
delay_real = jitter

```

Métodos que se reintentan

Por seguridad, solo se reintentan métodos idempotentes. La lista es **configurable por ruta** mediante `retryableMethods`.

Método	¿Por defecto?	Razón
GET	sí	Idempotente por definición
HEAD	sí	Igual a GET sin body
OPTIONS	sí	Sin side-effects
PUT	sí	Idempotente cuando el backend garantiza reemplazo total
DELETE	sí	Idempotente: borrar 2 veces = borrar 1 vez
POST	no	Suele crear recursos; reintentar duplica
PATCH	no	Aplica cambios incrementales

Ejemplo — habilitar retry para POST con idempotency-key

```

routes:
  - prefix: "/api/payments"
    target: "http://payments:8089"
    circuitBreaker:
      enabled: true
      maxRetries: 3
    retryableMethods:
      - POST
      - GET
      - PUT
      - DELETE
    # Solo si tu backend implementa idempotency-key (e.g. cabecera Idempotency-Key)

```

Errores elegibles para retry

- **Red:** ECONNREFUSED, ETIMEDOUT, ECONNRESET, ENOTFOUND, EPIPE
- **HTTP 5xx:** 500, 502, 503, 504

Los 4xx no se reintentan — son errores del cliente, no del backend.

NUNCA SE REINTENTA CON CIRCUIT ABIERTO

Si el circuit breaker está en estado OPEN, la request responde inmediatamente con 503 y nunca se reintenta.

12. Métricas Prometheus

El gateway expone un endpoint `GET /metrics` en formato Prometheus, incluyendo las métricas por defecto de Node.js y todas las internas del pipeline. Es consumido por Prometheus, Grafana y Dozzle.

Esquema de `metrics`

Campo	Tipo	Default	Descripción
<code>enabled</code>	boolean	true	Activa el endpoint
<code>path</code>	string	<code>/metrics</code>	Path del endpoint
<code>defaultLabels</code>	map string	<code>{}</code>	Labels aplicados a todas las métricas

Ejemplo A — etiquetar el entorno

```
metrics:
  enabled: true
  path: "/metrics"
  defaultLabels:
    environment: "production"
    service: "gateway"
    region: "us-east-1"
```

Métricas expuestas

Familia	Métrica	Labels
HTTP general	gateway_http_requests_total	route, backend, status_code
	gateway_http_request_duration_seconds	route, backend
	gateway_http_requests_in_flight	—
Rate limit	gateway_rate_limit_decisions_total	route, decision
	gateway_rate_limit_redis_errors_total	route
	gateway_rate_limit_window_seconds	route
Circuit breaker	gateway_circuit_breaker_state	route, backend (0=CLOSED, 1=HALF_OPEN, 2=OPEN)
	gateway_circuit_breaker_failures_total	route, backend
	gateway_circuit_breaker_requests_total	route, backend, status
	gateway_circuit_breaker_transitions_total	route, backend, from_state, to_state
	gateway_circuit_breaker_open_total	route, backend
Retries	gateway_retries_total	route, backend, error_code
	gateway_retry_success_total	route, backend
CORS	gateway_cors_requests_total	decision (allowed/blocked/preflight/no_origin)
JWT	gateway_jwt_validations_total	result
	gateway_jwks_refresh_total	result

Consultas PromQL de ejemplo

```

# Tasa de error 5xx por backend (últimos 5 min)
sum by (backend) (
  rate(gateway_http_requests_total{status_code=~"5.."}[5m])
)

# Porcentaje de preflights permitidos
sum(rate(gateway_cors_requests_total{decision="preflight"}[5m]))
/
sum(rate(gateway_cors_requests_total[5m])) * 100

# Auth Service caído (errores 503)
rate(gateway_jwt_validations_total{result="service_unavailable"}[5m])

# Latencia p95 por ruta
histogram_quantile(0.95,
  sum by (route, le) (
    rate(gateway_http_request_duration_seconds_bucket[5m])
  )
)

# Circuits actualmente abiertos
gateway_circuit_breaker_state == 2

```

13. Hot reload (SIGHUP)

El gateway puede recargar la configuración sin reiniciar el proceso enviando la señal `SIGHUP`. La actualización es atómica (swap de `ConfigSnapshot`) y los requests en vuelo terminan con la configuración anterior.

¿Cómo dispararlo?

```
# En un contenedor Docker
docker kill -s SIGHUP gateway-service

# En un proceso local con PID conocido
kill -s SIGHUP $(pgrep -f gateway-http)

# Desde systemd (si tienes un unit)
systemctl reload gateway
```

¿Qué se puede recargar en caliente?

Sección	¿Hot reload?	Comentario
<code>routes[].rateLimit</code>	✓ sí	Aplica en el siguiente request
<code>routes[].jwt</code>	✓ sí	El registry de JWKS se reconstruye
<code>routes[].cors</code>	✓ sí	Calcula nuevo <code>effectiveCors</code>
<code>overrides[]</code>	✓ sí	Cambio inmediato
<code>cors</code> + <code>corsOverrides[]</code>	✓ sí	Igual que por ruta
<code>jwt</code> + <code>jwtOverrides[]</code>	✓ sí	Registry se reconstruye con cooldown
<code>logging.level</code>	✓ sí	Cambio dinámico vía Pino
<code>server.port</code> / <code>server.host</code>	X no	Requiere reinicio (re-bind TCP)
<code>redis.url</code>	X no	Conexión Redis persiste
<code>routes[].prefix</code> / <code>target</code> / <code>stripPrefix</code>	X no	Cambio estructural
Añadir/quitar rutas	X no	Cambio estructural
<code>health.enabled</code>	X no	Registrado al startup

Comportamiento ante cambio no recargable

Si modificas `server.port` y envías SIGHUP, el gateway:

1. Detecta el cambio no recargable.
2. Lo registra como `warn` y lo ignora.
3. Continúa aplicando el resto de cambios válidos.
4. Devuelve en el log algo como: `config: reload partial – ignored non-reloadable: server.port`

Validación y rollback

Si la nueva configuración falla validación Zod:

1. El reload se descarta por completo.
2. El snapshot anterior se mantiene (rollback automático).
3. Se registra como `error` con detalle de los issues.

Protección contra señales concurrentes

Un SIGHUP mientras hay un reload en curso se ignora silenciosamente y se loguea como `warn`. No hay riesgo de race condition ni de double-swap de snapshots.

Ejemplo de flujo de cambio en producción

1. Editas `config/gateway.yaml` y subes el límite de `/api/login` de 5 a 20 req/min.
2. Mueves el archivo al volumen del contenedor (`docker cp`, sync, mount NFS).
3. `docker kill -s SIGHUP gateway-service`.
4. En `stdout` ves: `config: reloaded successfully (overrides: 1 path updated)`.
5. Desde el siguiente request, el nuevo límite está en vigor.

SOBRE WINDOWS

`SIGHUP` no se entrega de forma nativa en Windows. En entornos Windows el hot reload solo funciona dentro de WSL, Git Bash o contenedores Docker que lo traduzcan a la señal POSIX correcta.

14. Despliegue con Docker

El repositorio incluye un Dockerfile optimizado (multi-stage build) y un docker-compose de ejemplo con todo el stack de observabilidad: Redis, backend mock, Dozzle para logs en vivo, Prometheus y Grafana con un dashboard preinstalado.

Build de la imagen

```
# Usando el script de npm
pnpm docker:build

# Equivalente manual
docker build -f docker/Dockerfile -t gateway-http:latest .
```

Levantar el stack completo

```
docker compose -f docker/docker-compose.example.yml up --build
```

Componentes del stack

Servicio	Puerto local	Propósito
gateway	3000	El Gateway HTTP Modular
redis-cache	—	Redis como backend de rate-limit
mock-backend	—	Backend Node de prueba para requests de ejemplo
dozzle	9999	Visor de logs en tiempo real
prometheus	9090	Scraper de métricas
grafana	3001	Dashboard "Gateway Overview" pre-cargado

Hot reload en un contenedor

```
docker kill -s SIGHUP gateway-service
```

Ejemplo mínimo de `docker-compose.yml` de producción

```

services:
  gateway:
    image: gateway-http:latest
    ports:
      - "3000:3000"
    environment:
      CONFIG_PATH: /app/config/gateway.yaml
      REDIS_URL: redis://redis:6379
      NODE_ENV: production
    volumes:
      - ./config/gateway.yaml:/app/config/gateway.yaml:ro
    depends_on:
      redis:
        condition: service_healthy
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 3

```

Verificación rápida

```

# Health del gateway
curl -s http://localhost:3000/health | jq

# Métricas Prometheus – primeras 20 líneas
curl -s http://localhost:3000/metrics | head -20

# Logs en vivo: abrir http://localhost:9999 en el navegador

```

15. Solución de problemas

15.1 — El proceso no arranca y dice "*REDIS_URL is undefined*"

Causa: falta la variable de entorno `REDIS_URL` en el shell o en el bloque `environment` de Docker.

Solución:

```
# Linux/macOS
export REDIS_URL="redis://localhost:6379"

# PowerShell
$env:REDIS_URL = "redis://localhost:6379"
```

15.2 — "*cors: cannot use credentials=true with wildcard origin*"

Causa: el plugin CORS rechaza la combinación `credentials: true` + `origins: ["*"]` por estándar Fetch.

Solución: enumerar orígenes específicos o dejar `credentials: false`.

15.3 — Circuit breaker constantemente abierto

Síntoma: `gateway_circuit_breaker_state == 2` y muchas requests devuelven 503.

Pasos:

1. Revisar `gateway_http_requests_total` filtrado por status 5xx.
2. Comprobar que el backend upstream está saludable (`curl <target>/health` directo).
3. Revisar `recoveryTimeMs`; si es muy largo el backend tendrá más tiempo para recuperarse del estado OPEN.
4. Bajar `errorThreshold` temporalmente puede ser contraproducente — el problema suele ser que el threshold ya es muy sensible y se abre con tráfico legítimo.

15.4 — Las métricas no aparecen en Grafana

Pasos:

1. Confirmar que Prometheus hace scraping: abrir `http://localhost:9090/targets`.
2. Verificar que Grafana tiene configurado el data source apuntando a Prometheus.
3. Si usas `metrics.defaultLabels`, recordar que cambian el conjunto de series — los dashboards provisionados pueden necesitar actualización.

15.5 — "*jwt: unknown kid*" tras rotar claves en el Auth Service

Causa: el `cacheTtlSeconds` aún no expiró y la clave nueva no está en caché.

Solución:

- Verificar que `refreshOnMiss: true` en el issuer correspondiente.
- Reducir `cacheTtlSeconds` para que el refresh ocurra más seguido.
- Esperar `refreshCooldownSeconds` para evitar rate-limit por el cooldown y revisar los logs `gateway_jwks_refresh_total`.

15.6 — Hot reload no aplica cambios

Posibles causas:

- Estás modificando un campo **no recargable** (ver tabla en [cap. 13](#)).
- El archivo YAML está mal formado y la validación Zod falla — revisa los logs buscando `config: reload failed`.
- El contenedor Docker no tiene el volumen montado en `:ro` legible para el proceso.
- La señal `SIGHUP` no llegó porque el contenedor está usando PID 1 sin `tini` o `dumb-init`.

15.7 — Muchas requests devuelven 429

Pasos:

1. Inspeccionar `gateway_rate_limit_decisions_total` filtrado por `decision="blocked"`.
2. Ajustar `maxRequests` o `windowSeconds` en el override correspondiente.
3. Confirmar que detrás del gateway no haya un NAT corporativo que concentre muchas IPs reales en una sola.

15.8 — "MissingEnvVarError" en el log

Causa: una variable `${...}` en `gateway.yaml` no existe en el entorno del proceso.

Solución: añadir la variable al `.env` del despliegue, al `environment` del Dockerfile, o directamente como `env` en el manifiesto de Kubernetes.

15.9 — Health aggregation devuelve 503 aunque los servicios están OK

- Verificar que `backendPath` declarado en `health.backendPath` exista en cada microservicio.
- Si los servicios están en otro namespace de DNS, agregar el FQDN en `target` (ej: `http://auth-service.default.svc.cluster.local:8082`).
- Subir `timeoutMs` si el primer hit causa cold start (JVM, Node con lazy require, etc.).

Anexo: hoja de referencia rápida

Esqueleto completo

```

server:
  port: ${PORT}
  host: "0.0.0.0"

redis:
  url: "${REDIS_URL}"
  onFailure: "open"      # open | closed

logging:
  level: "info"          # debug | info | warn | error

metrics:
  enabled: true
  path: "/metrics"
  defaultLabels:
    environment: "production"

health:
  enabled: true
  path: "/health"
  backendPath: "/health"
  timeoutMs: 2000

cors:
  enabled: true
  origins: ["https://app.example.com"]
  methods: [GET, POST, PUT, DELETE, PATCH, OPTIONS]
  allowedHeaders: [Content-Type, Authorization]
  exposedHeaders: [X-Request-ID]
  credentials: false
  maxAge: 86400

jwt:
  enabled: true
  mode: "jwks"           # shared-secret | jwks
  issuers:
    - name: "primary"
      jwksUri: "https://auth.example.com/.well-known/jwks.json"
      issuer: "https://auth.example.com"
      audience: "my-api"

routes:
  - prefix: "/api/protected"
    target: "http://backend:3000"
    stripPrefix: true
    backendName: "backend"
    rateLimit:
      maxRequests: 1000
      windowSeconds: 60
    timeout:
      connect: 3000
      headers: 5000
      body: 30000

```

```

circuitBreaker:
  enabled: true
  errorThreshold: 50
  recoveryTimeMs: 30000
  halfOpenRequests: 3
  maxRetries: 3
  retryDelayMs: 100
  retryMaxDelayMs: 5000
jwt:
  issuer: "primary"

overrides:
- path: "/api/login"
  rateLimit:
    maxRequests: 5
    windowSeconds: 60

corsOverrides:
- path: "/api/auth/login"
  cors:
    origins: ["*"]

jwtOverrides:
- path: "/api/public"
  jwt:
    enabled: false

```

Señales operacionales

Señal	Efecto
<code>SIGHUP</code>	Relee <code>gateway.yaml</code> y aplica cambios hot-reloadables
<code>SIGTERM</code>	Shutdown ordenado: cierra Fastify → cierra Undici pools → quit Redis
<code>SIGINT</code>	Igual que SIGTERM (Ctrl+C en consola)

Endpoints reservados

Path	Tipo	Notas
<code>/metrics</code>	Prometheus	Configurable vía <code>metrics.path</code>
<code>/health</code>	JSON agregado	Configurable vía <code>health.path</code>

Métricas clave para alertas

Alerta	Consulta
Más del 5% de respuestas son 5xx	<code>sum(rate(gateway_http_requests_total{status_code=~"5.."}[5m])) / sum(rate(gateway_http_requests_total[5m])) > 0.05</code>
Circuit abierto más de 2 min	<code>changes(gateway_circuit_breaker_state[2m]) > 0 and gateway_circuit_breaker_state == 2</code>
Auth Service caído	<code>rate(gateway_jwt_validations_total{result="service_unavailable"}[1m]) > 0</code>
Latencia p95 > 2 s	<code>histogram_quantile(0.95, rate(gateway_http_request_duration_seconds_bucket[5m])) > 2</code>
Health degradado	<code>probe_success{job="gateway-health"} == 0</code> (si configuras blackbox exporter)

Gateway HTTP Modular · Manual de Usuario v1.0 · Julio 2026
Documento generado sobre la base de código en la rama actual del repositorio.